

**Web Basic**

---

# Web Basic

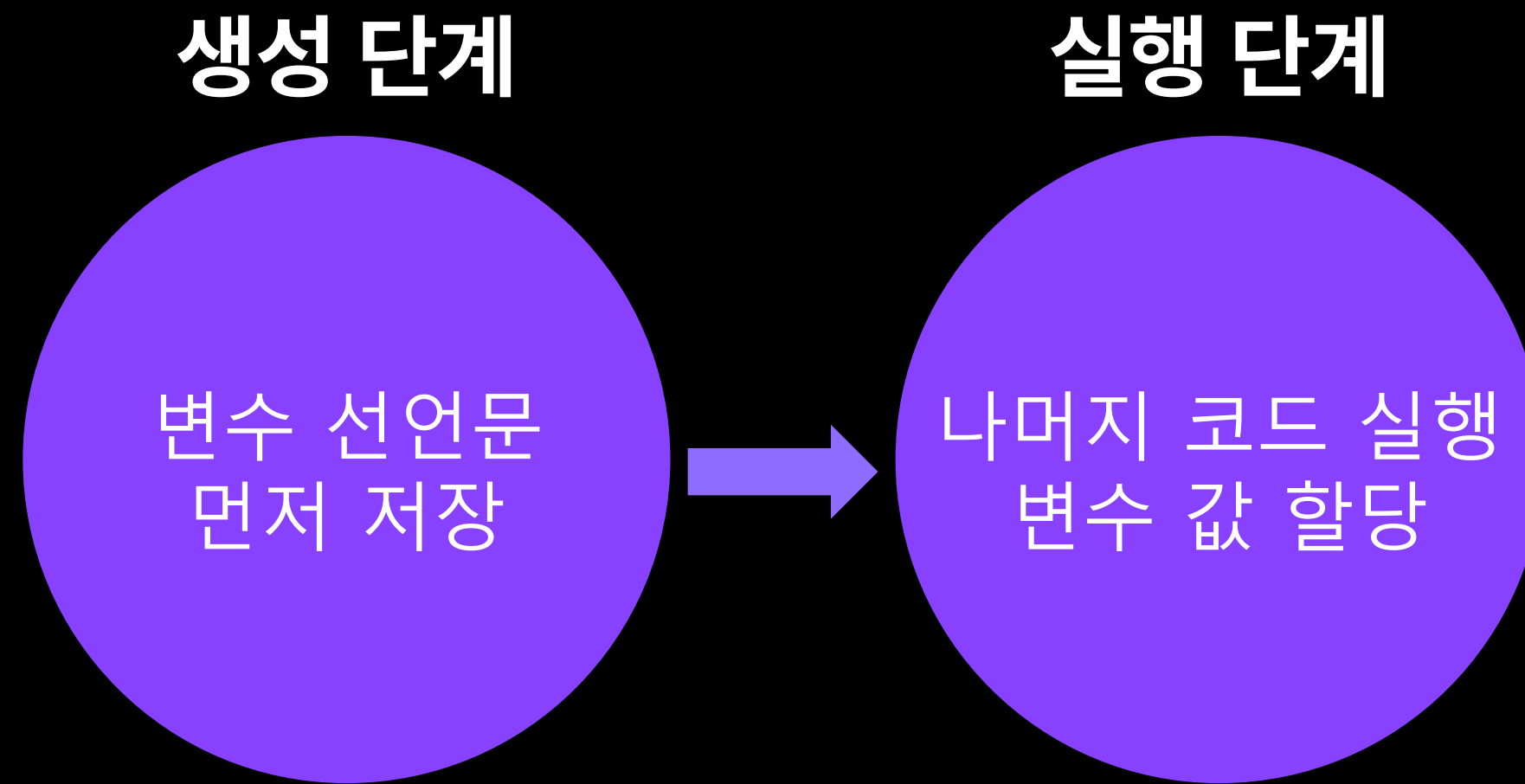
JavaScript 기초

# 실행 컨텍스트



작성한 자바스크립트 코드는 자바스크립트 엔진을 통해 파싱되고 실행된다.

가장 많이 사용하는 Chrome 브라우저의 경우 V8 엔진을 사용한다.



자바스크립트 엔진은 코드 실행 전 **실행 컨텍스트**라는 것을 생성한다.  
실행 컨텍스트는 위의 두 단계를 통해 생성 및 사용된다.





## 실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

전역 실행 컨텍스트 생성

그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.



# 실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

func1 실행 컨텍스트 생성



그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.



## 실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

func2 실행 컨텍스트 생성



그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.

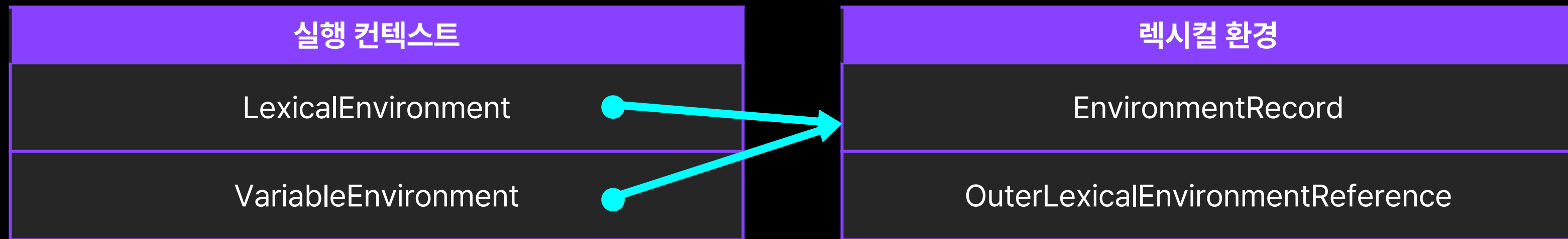
# 렉시컬 환경 (Lexical Environment)

| 실행 컨텍스트              |
|----------------------|
| Lexical Environment  |
| Variable Environment |

실행 컨텍스트는 **렉시컬 환경**으로 구성되어 있다.



# 렉시컬 환경 (Lexical Environment)



렉시컬 환경에는 해당 스코프에서의  
실질적인 변수 및 함수 선언, 스코프 체인에 대한 정보가 들어있다.

`EnvironmentRecord`: 변수 및 함수 선언과 값에 대한 정보

`OuterLexicalEnvironmentReference`: 상위 스코프의 렉시컬 환경에 대한 참조 (스코프 체인)



# 자바스크립트 코드 실행 단계

```
const a = 10;

if (a > 0) {
  const b = 20;
}

function func (c) {
  const d = 40;
}

func(30);
```

1. 전역 코드 평가 → 전역 실행 컨텍스트 & 렉시컬 환경 생성
2. 전역 코드 실행 → 전역 렉시컬 환경 업데이트
3. 블록 코드 평가 → 블록 렉시컬 환경 생성 및 연결
4. 블록 코드 실행 → 블록 렉시컬 환경 업데이트
5. 함수 코드 평가 → 함수 실행 컨텍스트 & 렉시컬 환경 생성
6. 함수 코드 실행 → 함수 렉시컬 환경 업데이트



# 자바스크립트 코드 실행 단계

```
const a = 10;
```

```
if (a > 0) {  
  const b = 20;  
}
```

```
function func (c) {  
  const d = 40;  
}
```

```
func(30);
```

전역 실행 컨텍스트

LexicalEnvironment

전역 렉시컬 환경

EnvironmentRecord

a

[초기화 안됨]

func

f func()

OuterLexicalEnvironmentReference

null

1. 전역 코드 평가 → 전역 실행 컨텍스트 & 렉시컬 환경 생성

: 변수 및 함수의 선언문을 먼저 실행시켜 렉시컬 환경에 기록



# 자바스크립트 코드 실행 단계

```
const a = 10;
```

```
if (a > 0) {  
  const b = 20;  
}
```

```
function func (c) {  
  const d = 40;  
}
```

```
func(30);
```

전역 실행 컨텍스트

LexicalEnvironment

전역 렉시컬 환경

EnvironmentRecord

a

[초기화 안됨]

func

f func()

this

OuterLexicalEnvironmentReference

null

1. 전역 코드 평가 → 전역 실행 컨텍스트 & 렉시컬 환경 생성  
: 추가로 **this 바인딩**을 진행 (추후 학습)





# 자바스크립트 코드 실행 단계



2. 전역 코드 실행 → 전역 렉시컬 환경 업데이트  
: 선언된 변수의 초기화 코드를 만나면 해당 값으로 업데이트





# 자바스크립트 코드 실행 단계

```
const a = 10;
```

```
if (a > 0) {  
  const b = 20;  
}
```

```
function func (c) {  
  const d = 40;  
}
```

```
func(30);
```

전역 실행 컨텍스트

LexicalEnvironment

블록 렉시컬 환경

EnvironmentRecord

b

[초기화 안됨]

OuterLexicalEnvironmentReference

전역 렉시컬 환경

EnvironmentRecord

a

10

func

f func()

this

OuterLexicalEnvironmentReference

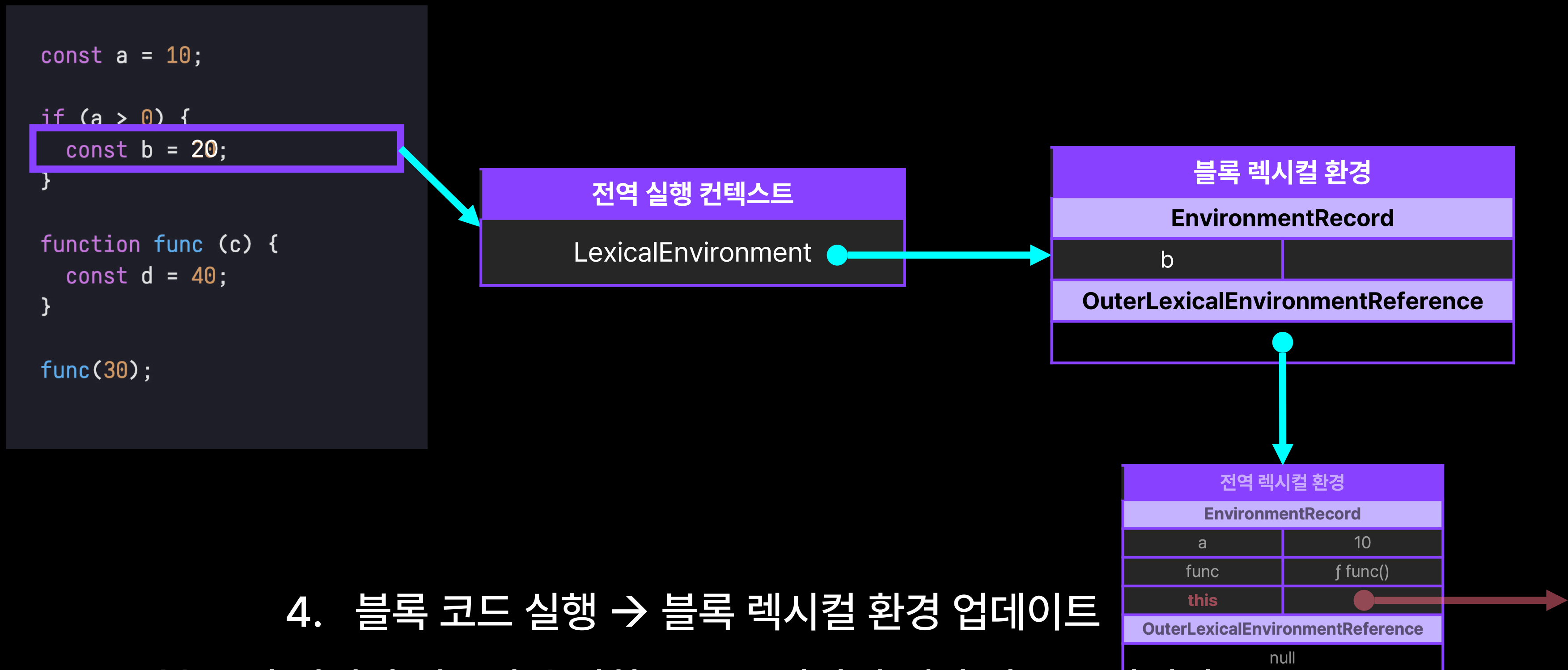
null

### 3. 블록 코드 평가 → 블록 렉시컬 환경 생성 및 연결

: 블록 실행 직전 블록 렉시컬 환경 생성, 현재 실행중인 실행 컨텍스트와 연결



# 자바스크립트 코드 실행 단계



## 4. 블록 코드 실행 → 블록 렉시컬 환경 업데이트

: 블록 내 선언된 변수의 초기화 코드를 만나면 해당 값으로 업데이트



# 자바스크립트 코드 실행 단계

```
const a = 10;  
  
if (a > 0) {  
  const b = 20;  
}
```

```
function func (c) {  
  const d = 40;  
}
```

```
func(30);
```

전역 실행 컨텍스트

LexicalEnvironment

전역 렉시컬 환경

EnvironmentRecord

|                                  |          |
|----------------------------------|----------|
| a                                | 10       |
| func                             | f func() |
| this                             |          |
| OuterLexicalEnvironmentReference |          |
| null                             |          |

블록 실행 이후에는 전역 실행 컨텍스트가  
다시 전역 렉시컬 환경에 연결된다.



# 자바스크립트 코드 실행 단계

```
const a = 10;  
  
if (a > 0) {  
  const b = 20;  
}
```

```
function func (c) {  
  const d = 40;  
}
```

```
func(30);
```

함수 func 실행 컨텍스트

LexicalEnvironment

함수 func 렉시컬 환경

EnvironmentRecord

|           |                                    |
|-----------|------------------------------------|
| c         | [초기화 안됨]                           |
| arguments | { 0: 30, length: 1, callee: func } |
| d         | [초기화 안됨]                           |

OuterLexicalEnvironmentReference

null

## 5. 함수 코드 평가 → 함수 실행 컨텍스트 & 렉시컬 환경 생성

: 함수 내의 선언문을 먼저 실행시켜 렉시컬 환경에 기록 (매개변수와 전달된 인수 포함)

전역 실행 컨텍스트

LexicalEnvironment

전역 렉시컬 환경

EnvironmentRecord

|   |    |
|---|----|
| a | 10 |
|---|----|





# 자바스크립트 코드 실행 단계

```
const a = 10;  
  
if (a > 0) {  
  const b = 20;  
}
```

```
function func (c) {  
  const d = 40;  
}
```

```
func(30);
```

함수 func 실행 컨텍스트

LexicalEnvironment

함수 func 렉시컬 환경

EnvironmentRecord

c

[초기화 안됨]

arguments

{ 0: 30, length: 1, callee: func }

d

[초기화 안됨]

this

OuterLexicalEnvironmentReference

5. 함수 코드 평가 → 함수 실행 컨텍스트 & 렉시컬 환경 생성

: 추가로 **this 바인딩**(추후 학습) 및 **상위 스코프 연결**을 진행

전역 실행 컨텍스트

LexicalEnvironment

전역 렉시컬 환경

EnvironmentRecord

a

10





# 자바스크립트 코드 실행 단계

```
const a = 10;
```

```
if (a > 0) {  
  const b = 20;  
}
```

```
function func (c) {  
  const d = 40;  
}
```

```
func(30);
```

함수 func 실행 컨텍스트

LexicalEnvironment

함수 func 렉시컬 환경

EnvironmentRecord

c

arguments

d

this

{ 0: 30, length: 1, callee: func }

OuterLexicalEnvironmentReference

## 6. 함수 코드 실행 → 함수 렉시컬 환경 업데이트

: 매개변수 업데이트 및 함수 내 선언된 변수의 초기화 코드를 만나면 해당 값으로 업데이트

전역 실행 컨텍스트

LexicalEnvironment

전역 렉시컬 환경

EnvironmentRecord

a

10



# 자바스크립트 코드 실행 단계

```
const a = 10;  
  
if (a > 0) {  
  const b = 20;  
}
```

```
function func (c) {  
  const d = 40;  
}
```

```
func(30);
```

전역 실행 컨텍스트

LexicalEnvironment

전역 렉시컬 환경

EnvironmentRecord

|                                  |          |
|----------------------------------|----------|
| a                                | 10       |
| func                             | f func() |
| this                             |          |
| OuterLexicalEnvironmentReference |          |
| null                             |          |

함수 실행 이후에는 해당 함수의 실행 컨텍스트가

실행 컨텍스트 스택에서 빠져나온다.



# 실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

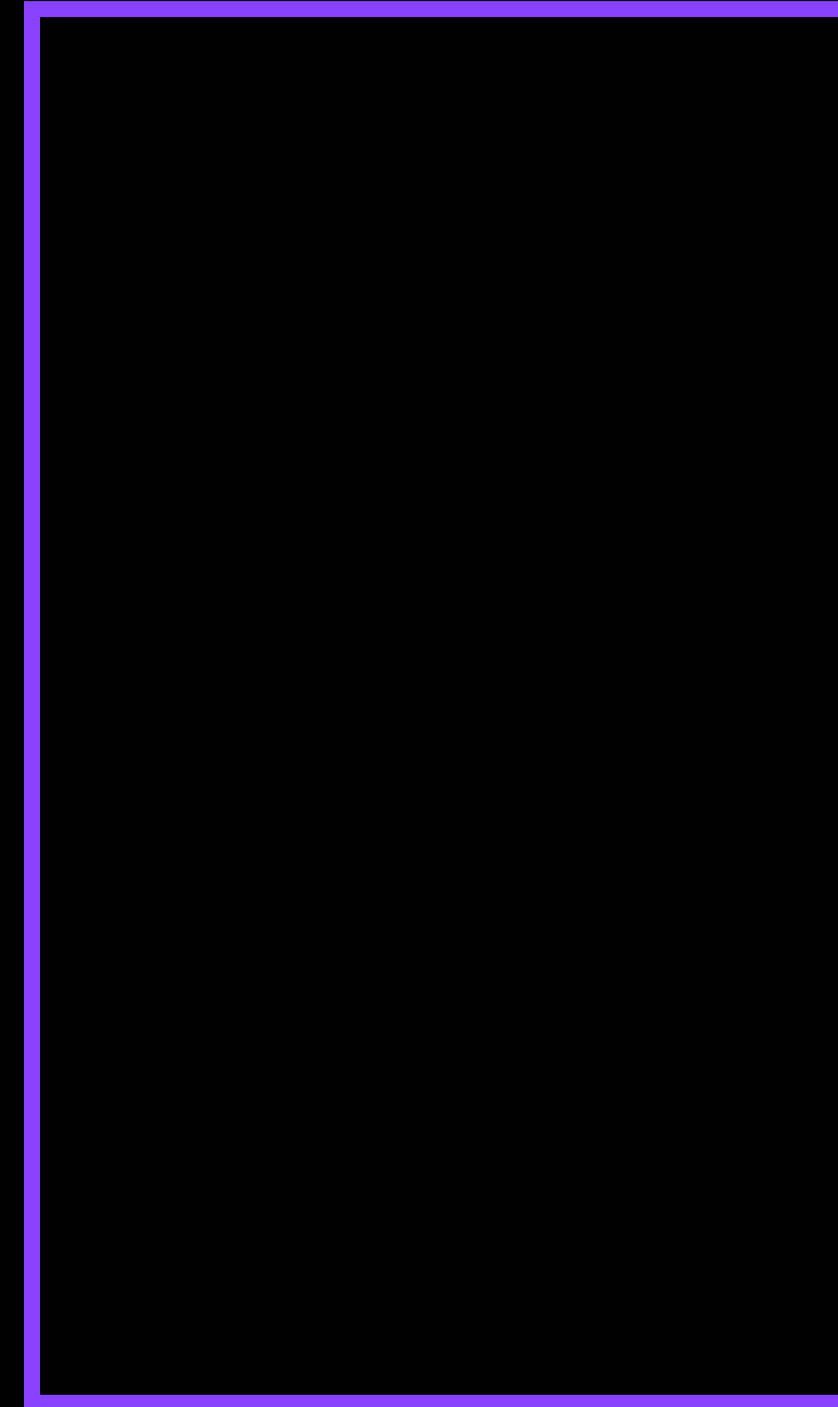
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



# 실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조





# 실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 outerFunc  
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조





# 실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 innerFunc  
실행 컨텍스트

함수 outerFunc  
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



# 실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 outerFunc  
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



# 실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



# 실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

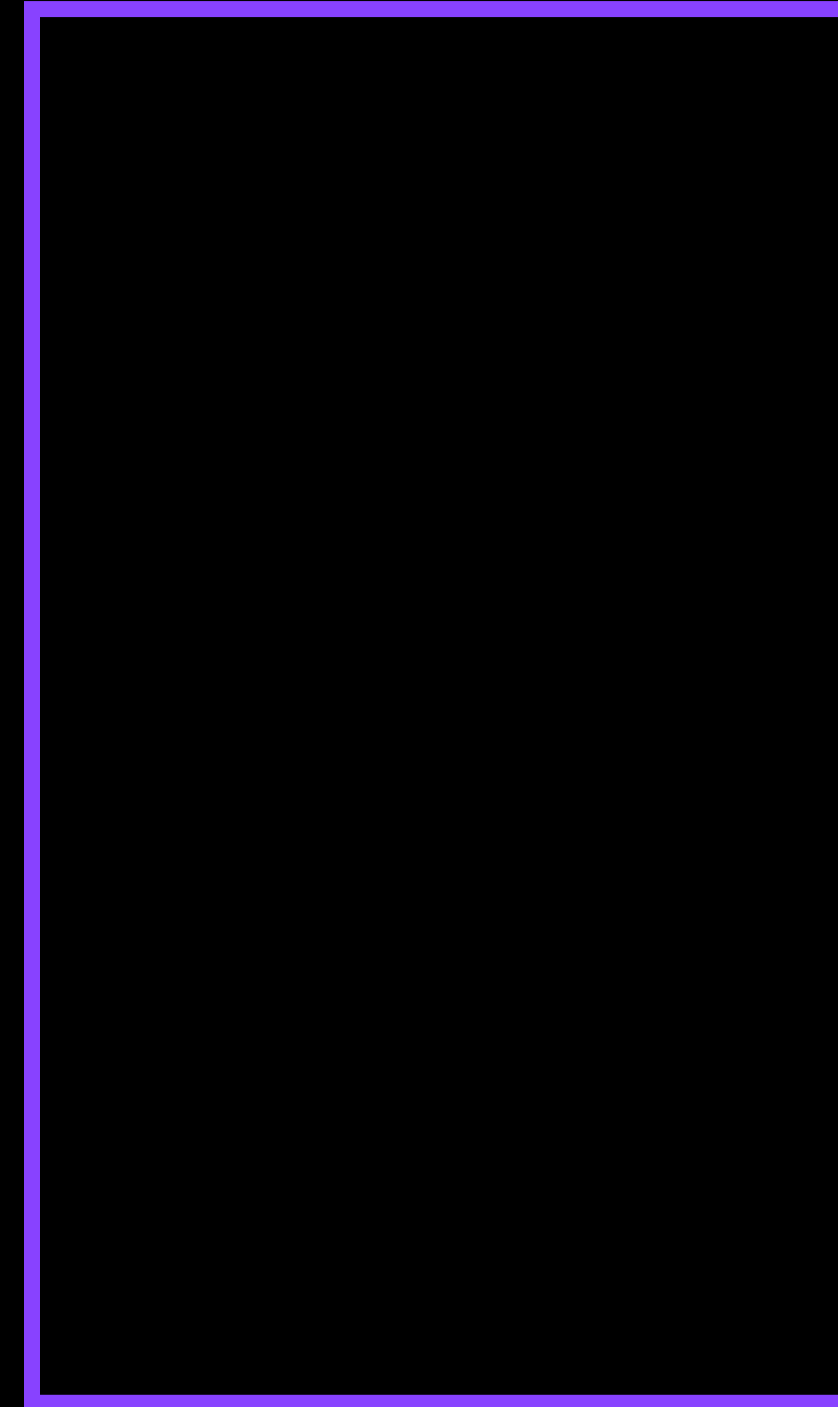
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



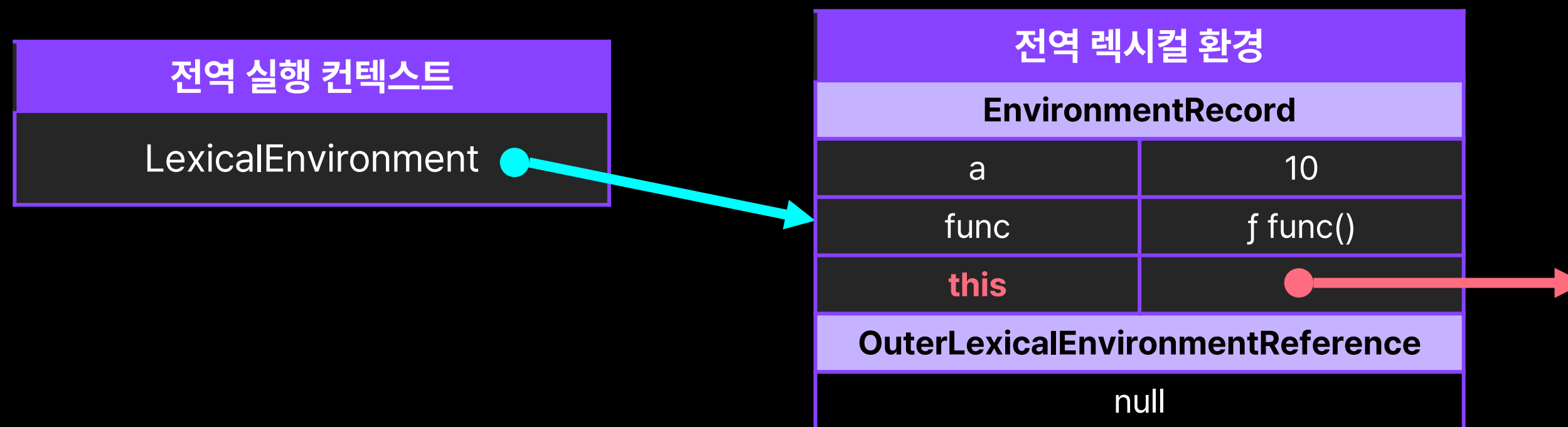
생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



# this와 변수 Hoisting





앞서 실행 컨텍스트의 도식에서 봤던 **this**란 무엇일까?

```
const hellobit = {  
  name: "hellobit",  
  age: 20,  
  introduce() {  
    console.log(`My name is ${    } and I am ${    } years old`);  
  }  
}  
  
hellobit.introduce();
```

객체 내부의 메서드에서, 해당 객체 자신의 프로퍼티를 참조하려면 어떻게 해야 할까?

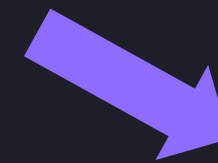
```
const hellobit = {  
  name: "hellobit",  
  age: 20,  
  introduce() {  
    console.log(`My name is ${name} and I am ${age} years old`);  
  }  
}  
  
hellobit.introduce();
```



ReferenceError: age is not defined

단순하게 프로퍼티의 이름을 사용하면,  
해당 메서드 내의 변수가 아니기 때문에 에러가 발생한다!  
(name에 대해 에러가 나지 않는 이유는 전역 객체인 window에 name이라는 프로퍼티가 존재하기 때문)

```
const hellobit = {  
  name: "hellobit",  
  age: 20,  
  introduce() {  
    console.log(`My name is ${hellobit.name} and I am ${hellobit.age} years old`);  
  }  
}  
  
hellobit.introduce();
```



'My name is hellobit and I am 20 years old'

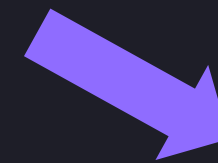
자기 자신의 객체명을 재귀적으로 사용하여 해결할 수 있다.

그러나 이는 바람직하지 않다.

(코드의 재사용성 감소, 테스트 어려움 등의 이유)



```
const hellobit = {  
  name: "hellobit",  
  age: 20,  
  introduce() {  
    console.log(`My name is ${this.name} and I am ${this.age} years old`);  
  }  
}  
  
hellobit.introduce();
```



```
'My name is hellobit and I am 20 years old'
```

이때 사용할 수 있는 것이 바로 **this** 식별자이다.

**this**는 자신이 속한 객체를 참조할 수 있는, **자기 참조 변수**이다.



**C++**

```
#include <iostream>
#include <string>

class Person {
private:
    std::string name;
    int age;

public:
    Person(const std::string& name, int age) {
        this->name = name;
        this->age = age;
    }

    void introduce() const {
        std::cout << "My name is " << this->name
        << " and I am " << this->age << " years old." <<
        std::endl;
    }
};

int main() {
    Person person("John", 30);
    person.introduce();
    return 0;
}
```

**Java**

```
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void introduce() {
        System.out.println("My name is " + this.name
        + " and I am " + this.age + " years old.");
    }

    public static void main(String[] args) {
        Person person = new Person("John", 30);
        person.introduce();
    }
}
```

다른 객체 지향 프로그래밍 언어에서도 this를 비슷한 의미로 사용한다.

다른 언어에서 this는 보통 **\*클래스 내부에서만** 사용할 수 있다.

```
const hellobit = {
  name: "hellobit",
  age: 20,
  introduce() {
    console.log(`My name is ${this.name} and I am ${this.age} years old`);
  }
}
hellobit.introduce();

console.log(this);

function func1() {
  console.log(this);
}
func1();
```

반면 JavaScript에서는 this를 어디에서나 사용할 수 있다.  
→ 대신 각 this가 **무엇과 바인딩되어 있는지** 파악해야 한다.



# this 바인딩 - 전역 및 일반 함수

```
console.log(this);
```

```
function func1() {  
  console.log(this);
```

```
  function func2() {  
    console.log(this);  
  }  
}
```

```
  func2();  
}
```

```
func1();
```

Window { ... }

Window { ... }

Window { ... }

기본적으로 this는 **전역 객체**와 바인딩 되어 있다.

일반적인 상황\*에서 전역 객체는 window 객체를 가리킨다.

\* strict mode 혹은 Node.js 환경에서는 전역 객체가 달라질 수 있습니다.



# this 바인딩 - 메서드

```
const hellobit = {  
  name: "hellobit",  
  age: 20,  
  introduce() {  
    console.log(`My name is ${this.name} and I am ${this.age} years old`);  
    console.log(this);  
  }  
}  
  
hellobit.introduce();
```



```
'My name is hellobit and I am 20 years old'  
{  
  name: 'hellobit',  
  age: 20,  
  introduce: f introduce()  
}
```

객체 내의 메서드에서는 this는 메서드를 호출한 객체와 바인딩 된다.



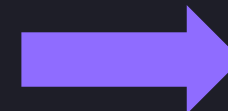


# this 바인딩 - 메서드

```
const hellobit = {  
  name: "hellobit",  
  age: 20,  
  introduce() {  
    console.log(`My name is ${this.name} and I am ${this.age} years old`);  
    console.log(this);  
  }  
}
```

```
const caterpillar = {  
  name: "caterpillar",  
  age: 23  
}
```

```
caterpillar.introduce = hellobit.introduce;  
caterpillar.introduce();
```



```
'My name is caterpillar and I am 23 years old'  
{  
  name: 'caterpillar',  
  age: 23,  
  introduce: f introduce()  
}
```

**호출한** 객체가 중요하기 때문에,

위와 같이 다른 객체에서 해당 메서드를 호출하면 해당 객체와 바인딩 된다.



# this 바인딩 – 간접 호출 (apply, call, bind)

```
const hellobit = {
  name: "hellobit",
  age: 20
}

const caterpillar = {
  name: "caterpillar",
  age: 23
}

const cheshire = {
  name: "cheshire",
  age: 26
}

function introduce() {
  console.log(`My name is ${this.name} and I am ${this.age} years old`);
  console.log(this);
}

introduce.apply(hellobit);
introduce.call(caterpillar);
introduce.bind(cheshire)();
```

```
'My name is hellobit and I am 20 years old'
{ name: 'hellobit', age: 20 }
'My name is caterpillar and I am 23 years old'
{ name: 'caterpillar', age: 23 }
'My name is cheshire and I am 26 years old'
{ name: 'cheshire', age: 26 }
```

‘apply’, ‘call’, ‘bind’ 메서드를 사용하면 함수를 **간접적으로 호출**할 수 있다.

이 방법을 통해 특정 객체와 함수를 **직접 바인딩**할 수 있다.

# this 바인딩 – 간접 호출 (apply, call, bind)

```
const hellobit = {
  name: "hellobit",
  age: 20
}

function printThisWithArgs(a, b) {
  console.log(this, a, b);
}

printThisWithArgs.apply(hellobit, [1, 2]); // apply: 배열로 인수 전달
printThisWithArgs.call(hellobit, 3, 4);   // call: 매개변수로 인수 전달
printThisWithArgs.bind(hellobit)(5, 6);   // bind: 바인딩 된 함수 자체를 반환
```



```
{ name: 'hellobit', age: 20 } 1 2
{ name: 'hellobit', age: 20 } 3 4
{ name: 'hellobit', age: 20 } 5 6
```



```
const btn = document.getElementById("submit-btn");  
btn.addEventListener("click", function () {  
  alert("button clicked");  
})
```

**콜백 함수**란 매개변수를 통해 다른 함수로 전달되는 함수를 의미한다.

(사용 예시: 이벤트 핸들러, 비동기 처리 (추후 학습), 배열 메서드 등)



## 콜백 함수 예시

```
function plus(a, b) { return a + b; }  
function minus(a, b) { return a - b; }  
  
function calculateAndSquare(a, b, calFunc) {  
  const result = calFunc(a, b);  
  return result * result;  
}  
  
calculateAndSquare(3, 5, plus);    // 64  
calculateAndSquare(3, 5, minus);  // 4
```





# this 바인딩과 콜백 함수

```
function calculateAndSquare(calFunc) {  
  const result = calFunc();  
  return result * result;  
}  
  
const calculator1 = {  
  a: 3,  
  b: 5,  
  calculate() {  
    return calculateAndSquare(function () {  
      return this.a + this.b;  
    });  
  }  
};  
  
calculator1.calculate(); // NaN
```

메서드 내에서 콜백 함수를 전달하면,  
해당 콜백 함수는 어떤 객체와 바인딩 될까?





# this 바인딩과 콜백 함수

```
function calculateAndSquare(calFunc) {  
  const result = calFunc();  
  return result * result;  
}  
  
const calculator1 = {  
  a: 3,  
  b: 5,  
  calculate() {  
    return calculateAndSquare(function () {  
      return this.a + this.b;  
    });  
  }  
};  
  
calculator1.calculate(); // NaN
```

해당 콜백 함수는 결국 특정 객체와 바인딩된 적이 없는 일반 함수이므로,  
**전역 객체와 바인딩 된다!**



# this 바인딩과 콜백 함수

```
function calculateAndSquare(calFunc) {  
  const result = calFunc();  
  return result * result;  
}  
  
const calculator1 = {  
  a: 3,  
  b: 5,  
  calculate() {  
    return calculateAndSquare((function () {  
      return this.a + this.b;  
    }).bind(this));  
  }  
};  
  
calculator1.calculate(); // 64
```

이를 해결하기 위해 `bind` 메서드를 사용하여  
**this와 직접 바인딩한다.**



# this 바인딩과 콜백 함수

```
function calculateAndSquare(calFunc) {  
  const result = calFunc();  
  return result * result;  
}  
  
const calculator1 = {  
  a: 3,  
  b: 5,  
  calculate() {  
    return calculateAndSquare(() => {  
      return this.a + this.b;  
    });  
  }  
};  
  
calculator1.calculate(); // 64
```

혹은 직접 바인딩 없이 **화살표 함수**를 활용하는 방법도 있다.



## 화살표 함수

```
function hello() { ... }      // 일반 함수
const hello = () => { ... };  // 화살표 함수

btn.addEventListener("click", function () { ... }); // 일반 함수
btn.addEventListener("click", () => { ... });        // 화살표 함수

function (a, b) { return a + b; }
(a, b) => a + b; // 중괄호 없이 작성하면 return 키워드를 생략하고 값을 반환할 수 있다.
```

화살표 함수는 ES6 이후에 도입된 새로운 함수 선언 형태로,  
일반 함수보다 **간략하게 함수를 선언**할 수 있다.





## 화살표 함수

```
function hello() { ... }      // 일반 함수
const hello = () => { ... };  // 화살표 함수

btn.addEventListener("click", function () { ... }); // 일반 함수
btn.addEventListener("click", () => { ... });        // 화살표 함수

function (a, b) { return a + b; }
(a, b) => a + b; // 중괄호 없이 작성하면 return 키워드를 생략하고 값을 반환할 수 있다.
```

일반 함수와 단순히 선언 방식만 다른 것이 아닌,

**this 바인딩 방식이 다르다.**

(이 외에도 일반 함수와 여러 차이점이 존재)





# 화살표 함수의 this

```
const inGlobal = () => { console.log(this); }

const obj = {
  inObj: () => { console.log(this); },
  method() {
    const inMethod = () => { console.log(this); }
    inMethod();
  }
};

inGlobal(); // Window { ... }
obj.inObj(); // Window { ... }

// inMethod의 상위 스코프는 method 메서드 -> method 메서드의 this를 참조한다!
// obj.method() 실행 시 this는 obj 객체와 바인딩 됨
obj.method(); // { inObj: f inObj(), method: f method() }
```

화살표 함수는 this 바인딩 자체가 존재하지 않는다.

따라서 화살표 함수 내부의 this는 그대로 상위 스코프의 this를 참조한다.



# 화살표 함수의 this

```
const inGlobal = () => { console.log(this); }

const obj = {
  inObj: () => { console.log(this); },
  method() {
    const inMethod = () => { console.log(this); }
    inMethod();
  }
};

inGlobal(); // Window { ... }
obj.inObj(); // Window { ... }

// inMethod의 상위 스코프는 method 메서드 -> method 메서드의 this를 참조한다!
// obj.method() 실행 시 this는 obj 객체와 바인딩 됨
obj.method(); // { inObj: f inObj(), method: f method() }
```

결국 화살표 함수가 **정의된 위치에 의해 결정**된다고 볼 수 있다.

→ 렉시컬 this



## 화살표 함수와 this 바인딩 문제

```
function calculateAndSquare(calFunc) {  
  const result = calFunc();  
  return result * result;  
}  
  
const calculator1 = {  
  a: 3,  
  b: 5,  
  calculate() {  
    return calculateAndSquare(() => {  
      return this.a + this.b;  
    });  
  }  
};  
  
calculator1.calculate(); // 64
```

앞선 바인딩 문제를 화살표 함수가 해결하는 이유는  
상위 스코프인 calculate의 this를 그대로 참조하기 때문이다.



## 변수 호이스팅

```
console.log("선언 전", elice);  
var elice = "hello";  
console.log("선언 후", elice);
```

위와 같이 변수 `elice`를 선언하기 전에 참조하면  
어떤 결과가 나타날까?





## 변수 호이스팅

```
console.log("선언 전", elice);  
var elice = "hello";  
console.log("선언 후", elice);
```

```
'선언 전' undefined  
'선언 후' 'hello'
```

에러가 발생할 것을 예상할 수 있지만, 실제로는 일어나지 않는다.





## 변수 호이스팅

```
console.log("선언 전", elice);  
var elice = "hello";  
console.log("선언 후", elice);
```

```
'선언 전' undefined  
'선언 후' 'hello'
```

이처럼 JavaScript에서 변수 선언이  
해당 스코프의 최상위로 끌어올려지는 현상을 **변수 호이스팅**이라고 한다.



# 변수 호이스팅의 이유

자바스크립트 코드 실행 단계

Web Basic

O2 JavaScript 기초 -  
보충자료

1. 실행 컨텍스트

```
const a = 10;

if (a > 0) {
  const b = 20;
}

function func (c) {
  const d = 40;
}

func(30);
```

1. 전역 코드 **평가** → 전역 실행 컨텍스트 & **렉시컬 환경 생성**
2. 전역 코드 실행 → 전역 렉시컬 환경 업데이트
3. 블록 코드 **평가** → 블록 **렉시컬 환경 생성** 및 연결
4. 블록 코드 실행 → 블록 렉시컬 환경 업데이트
5. 함수 코드 **평가** → 함수 실행 컨텍스트 & **렉시컬 환경 생성**
6. 함수 코드 실행 → 함수 렉시컬 환경 업데이트

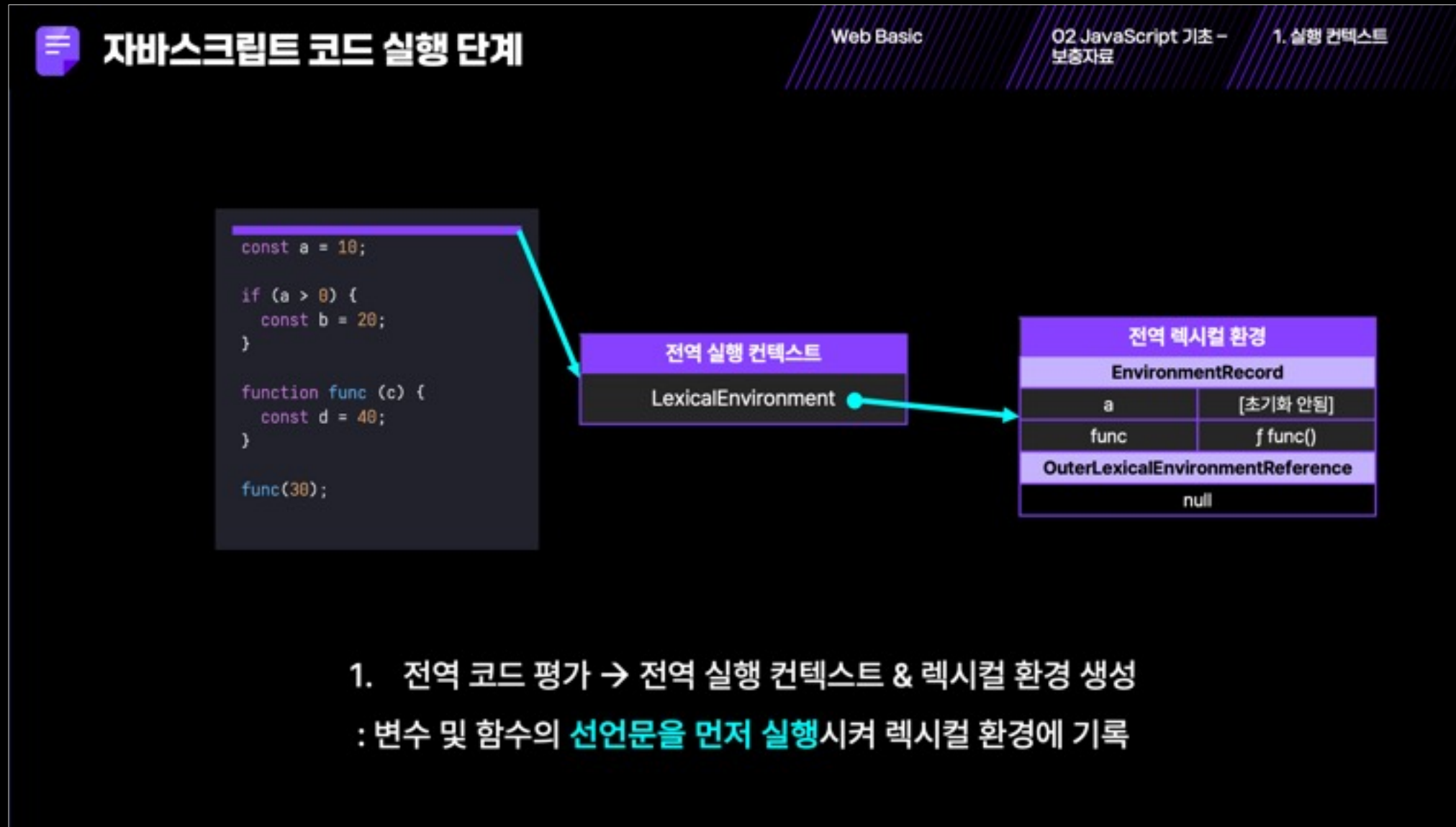
이러한 현상이 나타나는 이유는 실행 컨텍스트와 연관이 있다.

모든 스코프의 실행 전에는 **코드를 평가하는 단계**가 있고,

평가 과정에서 **실행 컨텍스트 및 렉시컬 환경이 생성**된다.



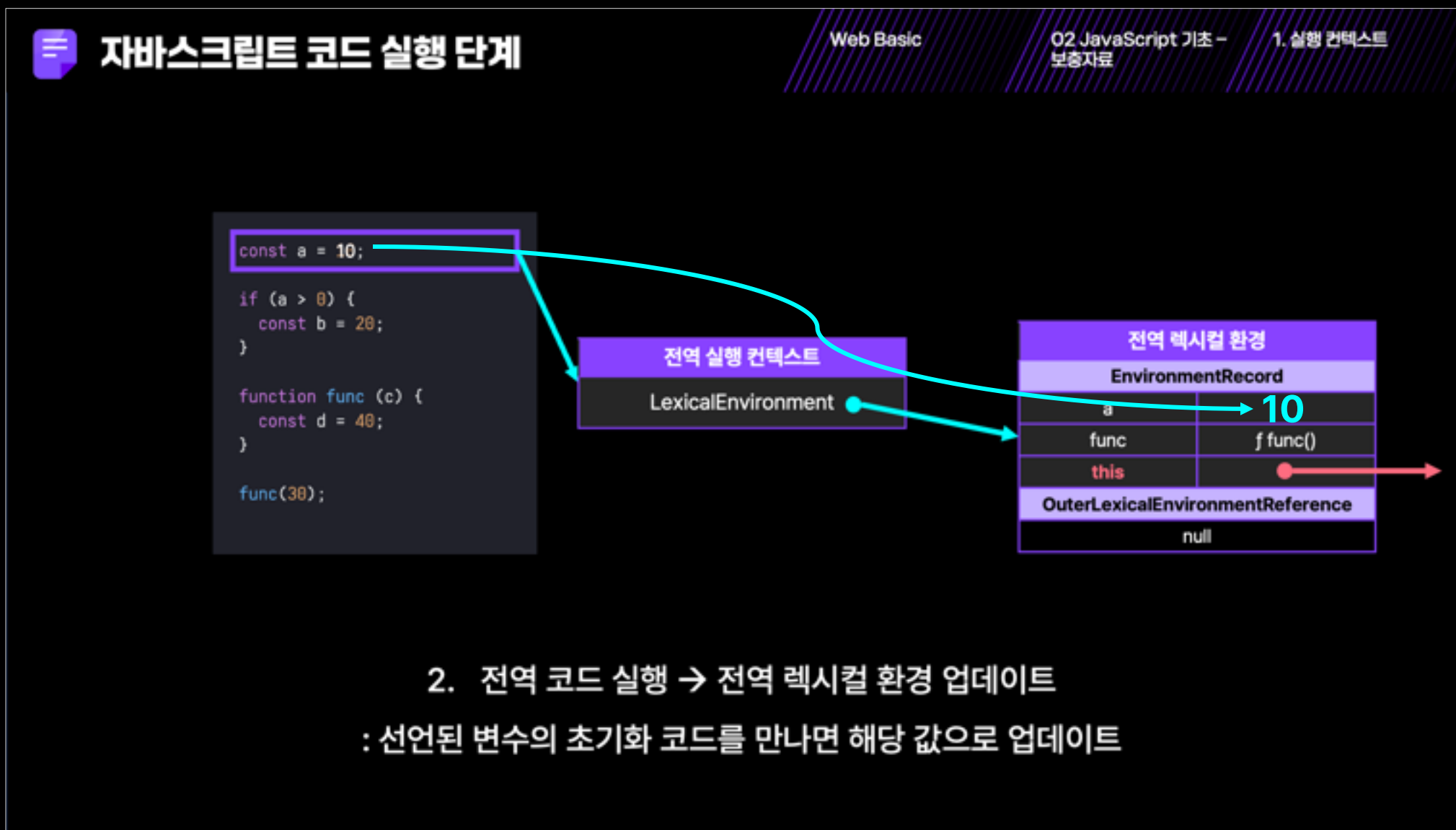
# 변수 호이스팅의 이유



이때 모든 변수와 함수의 선언문이 먼저 실행되고,  
렉시컬 환경의 EnvironmentRecord에 기록된다.



# 변수 호이스팅의 이유



그 이후에 코드가 한 줄씩 실행되면서  
초기화문에 도달하면 값을 할당한다.





## 변수 호이스팅의 이유

```
console.log("선언 전", elice);  
var elice = "hello";  
console.log("선언 후", elice);
```

### 전역 렉시컬 환경

#### EnvironmentRecord

elice

undefined

#### OuterLexicalEnvironmentReference

null

즉, 함수 선언문 이전에 이미 렉시컬 환경에 해당 변수가 등록되어 있다.





## 변수 호이스팅의 이유

```
console.log("선언 전", elice);  
var elice = "hello";  
console.log("선언 후", elice);
```

### 전역 렉시컬 환경

#### EnvironmentRecord

elice

undefined

#### OuterLexicalEnvironmentReference

null

‘var’ 키워드로 변수를 선언한 경우, 렉시컬 환경 등록 이후  
자바스크립트 엔진은 **해당 변수를 undefined로 초기화**한다.  
→ 따라서 선언문 이전에 참조해도 에러가 발생하지 않는다.

# var와 let, const의 차이점 - 변수 호이스팅

```
console.log("선언 전", elice); // ReferenceError
let elice = "hello";
console.log("선언 후", elice);
```

| 전역 렉시컬 환경                        |          |
|----------------------------------|----------|
| EnvironmentRecord                |          |
| elice                            | [초기화 안됨] |
| OuterLexicalEnvironmentReference |          |
| null                             |          |

반면 동일한 코드에서 `let`, `const`로 변수를 선언하면 에러가 발생한다.  
`let`, `const`는 렉시컬 환경 등록 이후에 **바로 초기화를 진행하지 않기** 때문이다.

# var와 let, const의 차이점 – 변수 호이스팅

```
console.log("선언 전", elice); // ReferenceError
let elice = "hello";
console.log("선언 후", elice);
```

| 전역 렉시컬 환경                        |          |
|----------------------------------|----------|
| EnvironmentRecord                |          |
| elice                            | [초기화 안됨] |
| OuterLexicalEnvironmentReference |          |
| null                             |          |

따라서 선언문이 먼저 실행되기 때문에 변수 호이스팅이 일어나긴 하지만,  
선언문 전에 참조하면 에러가 발생하므로 **변수 호이스팅이 일어나지 않는 것처럼** 보인다.

변수 선언 (렉시컬 환경 등록)

TDZ (Temporal Dead Zone)

```
console.log(선언 전, elice); // ReferenceError
```

```
let elice = "hello";  
console.log("선언 후", elice);
```

선언문 실행과 초기화 사이에 변수를 참조할 수 없는 구간을

**TDZ (Temporal Dead Zone)**이라고 부른다.